

ALGORITMI STUDIATI:

ARRAY:

```
binarySearch(<key, data> A[1..x], int n, key K)          O(logn)
|   i=1
|   j=n
|   while(i<=j)
|   |       m=(i+j)/2
|   |       if (A[m].key==K)
|   |       |       return m
|   |       else if (A[m].key<K)
|   |       |       i=m+1
|   |       else
|   |       |       j=m-1
|   |       return -1
```

LISTE:

```
searchNode(list L, data D)->Node          O(logn)
|   temp=L.head
|   while(temp!=NULL)
|   |       if (temp.data==D)
|   |       |       return temp
|   |       temp=temp.next
|   return NULL
```

```
insert(list L, data D)                     O(1)
|   temp=node(D)
|   if (L.head==NULL)
|   |       L.head=L.tail=temp
|   else
|   |       temp.next=L.head
|   |       L.head=temp
```

```
delete(list L, data D)                     O(n)
|   prev=NULL
|   curr=L.head
|   while(curr!=NULL && D!=curr.data)
|   |       prev=curr
|   |       curr=curr.next
|   if (curr!=NULL)
|   |       if (curr==L.head)
|   |       |       L.head=curr.next
|   |       else
|   |       |       prev.next=curr.next
|   |       if (curr==L.tail)
|   |       |       L.tail=prev
```

ALBERI:

```

postOrder(node T)                                O(n)
|      if (T!=NULL)
|      |      postOrder(T.left)
|      |      postOrder(T.right)
|      |      visit(T)

```

```

inOrder(node T)                                  O(n)
|      if (T!=NULL)
|      |      inOrder(T.left)
|      |      visit(T)
|      |      inOrder(T.right)

```

```

preOrder(node T)                                O(n)
|      if (T!=NULL)
|      |      visit(T)
|      |      preOrder(T.left)
|      |      preOrder(T.right)

```

```

BFS(tree T)                                      O(n)
|      Q=new Queue
|      if (T.root!=NULL)
|      |      Q.enqueue(T.root)
|      while (Q.size!=0)
|      |      x=Q.dequeue()
|      |      visit(x)
|      |      if (x.left!=NULL)
|      |      |      Q.enqueue(x.left)
|      |      if (x.right!=NULL)
|      |      |      Q.enqueue(x.right)

```

GRAFI:

BFS(graph $G=(V,E)$, vertex s) $\rightarrow$ tree Liste Adiacenza: $O(n+m)$; Matrice: $O(n^2)$

```

|      for each v in V
|      |      v.mark=false
|      T=s
|      F=new Queue
|      F.enqueue(s)
|      s.mark=true
|      s.dist=0
|      while (F.size!=0)
|      |      u=F.dequeue()
|      |      visit(u)
|      |      for each v adiacente a u
|      |      |      if (!v.mark)
|      |      |      |      v.mark=true
|      |      |      |      T=T union v
|      |      |      |      F.enqueue(v)
|      |      |      |      v.parent=u
|      |      |      |      v.dist=u.dist+1
|      return T

```

```
global int time=0;
```

```
DFS(graph G=(V,E))
```

Liste Adiacenza: $O(n+m)$; Matrice:

$O(n^2)$

```
|   for each v in V
|   |       v.mark=inesplorato
|   |       v.parent=NULL
|   for each v in V
|   |       if(v.mark==inesplorato)
|   |       |       DFSvisit(v)
```

```
DFSvisit(vertex u)
```

Liste Adiacenza: $O(\text{grado}(u))$;

Matrice: $O(m)$

```
|   u.mark=aperto
|   time++
|   u.discoveryTime=time
|   for each v adiacente a u
|   |       if(v.mark==inesplorato)
|   |       |       v.parent=u
|   |       |       DFSvisit(v)
|   visit(u)
|   time++
|   u.finishTime=time
|   u.mark=chiuso
```

```
kruskalMST(graph G=(V,E,w)) -> tree
```

$O(m \log n)$ $m \geq n-1$

```
|   UF = new node unionFind
|   T = new tree
|   for(i=1..n)
|   |       UF.makeSet(i)
|   sort(E,w)
|   for each {u,v} in E
|   |       uRoot=UF.find(u)
|   |       vRoot=UF.find(v)
|   |       if(uRoot!=vRoot)
|   |       |       T=T union {u,v}
|   |       |       UF.union(uRoot, vRoot)
|   return T
```

```

primMST(graph G=(V,E,w), node s)          O(mlogn) m>=n-1
|   for each v in V
|   |   v.dist=+inf
|   |   v.parent=NULL
|   |   v.bool=false
|   s.dist=0
|   Q = new <int,real> priority queue
|   Q.insert(s,s.dist)
|   while(!Q.isempty())
|   |   u=Q.findMin()
|   |   Q.deleteMin()
|   |   u.bool=true
|   |   for each v adiacente a u
|   |   |   if(!v.bool)
|   |   |   |   if(v.dist==+inf)
|   |   |   |   |   Q.insert(v,w(u,v))
|   |   |   |   |   v.dist=w(u,v)
|   |   |   |   |   v.parent=u
|   |   |   |   else if(w(u,v)<v.dist)
|   |   |   |   |   Q.decreaseKey(v,w(u,v))
|   |   |   |   |   v.dist=w(u,v)
|   |   |   |   |   v.parent=u

```

```

bellmanFordSingleSourceShortestPath(graph(V,E,w), int s)->real[1..n]
    O(mn)
|   int n=G.vertexNum();
|   int pred[1..n]
|   int v
|   int u
|   double D[1..n]
|   for(v=1..n)
|   |   D[v]=+inf
|   |   pred[v]=-1
|   D[s]=0
|   for(i=1..n-1)
|   |   for each (u,v) in E
|   |   |   if(D[v]>D[u]+w(u,v))
|   |   |   |   D[v]=D[u]+w(u,v)
|   |   |   |   pred[v]=u
|   for each (u,v) in E
|   |   if(D[v]>D[u]+w(u,v))
|   |   |   error(cicli di peso negativo)
|   return D;

```

```

dijkstraSingleSourceShortestPath(graph G=(V,E,w), int s)->real[1..n]
    O(mlogn)

```

```

|     int n=G.vertexNum()
|     int pred[1..n]
|     int v
|     int u
|     double D[1..n]
|     for(v=1..n)
|         |         D[v]=+inf
|         |         pred[v]=-1
|     D[s]=0
|     Q = new <int,real> priority queue
|     Q.insert(s,D[s])
|     while(!Q.isEmpty())
|         |         u=Q.find()
|         |         Q.deleteMin()
|         |         for each v adiacente a u
|         |             |         if (D[v]==+inf)
|         |             |             |         D[v]=D[u]+w(u,v)
|         |             |             |         Q.insert(v,D[v])
|         |             |             |         pred[v]=u
|         |             |         else if (D[v]>D[u]+w(u,v))
|         |             |             |         D[v]=D[u]+w(u,v)
|         |             |             |         pred[v]=u
|     return D

```

floydWarshallAllPairsShortestPath(graph G=(V,E,w)) -> real[1..n][1..n]

```

|         O(n^3)
|     int n=G.vertexNum()
|     D = new real [1..n][1..n] matrix
|     int x
|     int y
|     int k
|     next = new int [1..n][1..n] matrix
|     for(x=1..n)
|         |         for(y=1..n)
|         |             |         if (x==y)
|         |             |             |         D[x][y]=0
|         |             |             |         next[x][y]=-1
|         |             |         else if (x,y) in E
|         |             |             |         D[x][y]=w(x,y)
|         |             |             |         next[x][y]=y
|         |             |         else
|         |             |             |         D[x][y]=+inf
|         |             |             |         next[x][y]=-1
|     for(k=1..n)
|         |         for(x=1..n)
|         |             |         for(y=1..n)
|         |             |             |         if (D[x][y]>D[x][k]+D[k][y])
|         |             |             |             |         D[x][y]=D[x][k]+D[k][y]
|         |             |             |             |         next[x][y]=next[x][k]
|     for(x=1..n)
|         |         if (D[x][x]<0)
|         |             |         error(cicli di peso negativo)
|     return D

```

```

printPath(int u, int v, int next[1..n][1..n])      O(|E(u,v)|)
|   if(u!=v && next[u][v]<0)
|   |   error(u e v non connessi)
|   else
|   |   print(u)
|   |   while(u!=v)
|   |   |   u=next[u][v]
|   |   |   print(u)

```